

SPA con React

1. Introducción a React



Taxonomía de las webs

Existen varios tipos de webs

Dependiendo de cómo usan las tecnologías
de **cliente** y **servidor**

Página web

Servidor estático

El servidor web sirve
contenido guardado en
el disco duro

Aplicación web

Servidor dinámico

El servidor web sirve
contenido generado
mediante código

Página web

Servidor estático

- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

Taxonomía de las webs

Página web

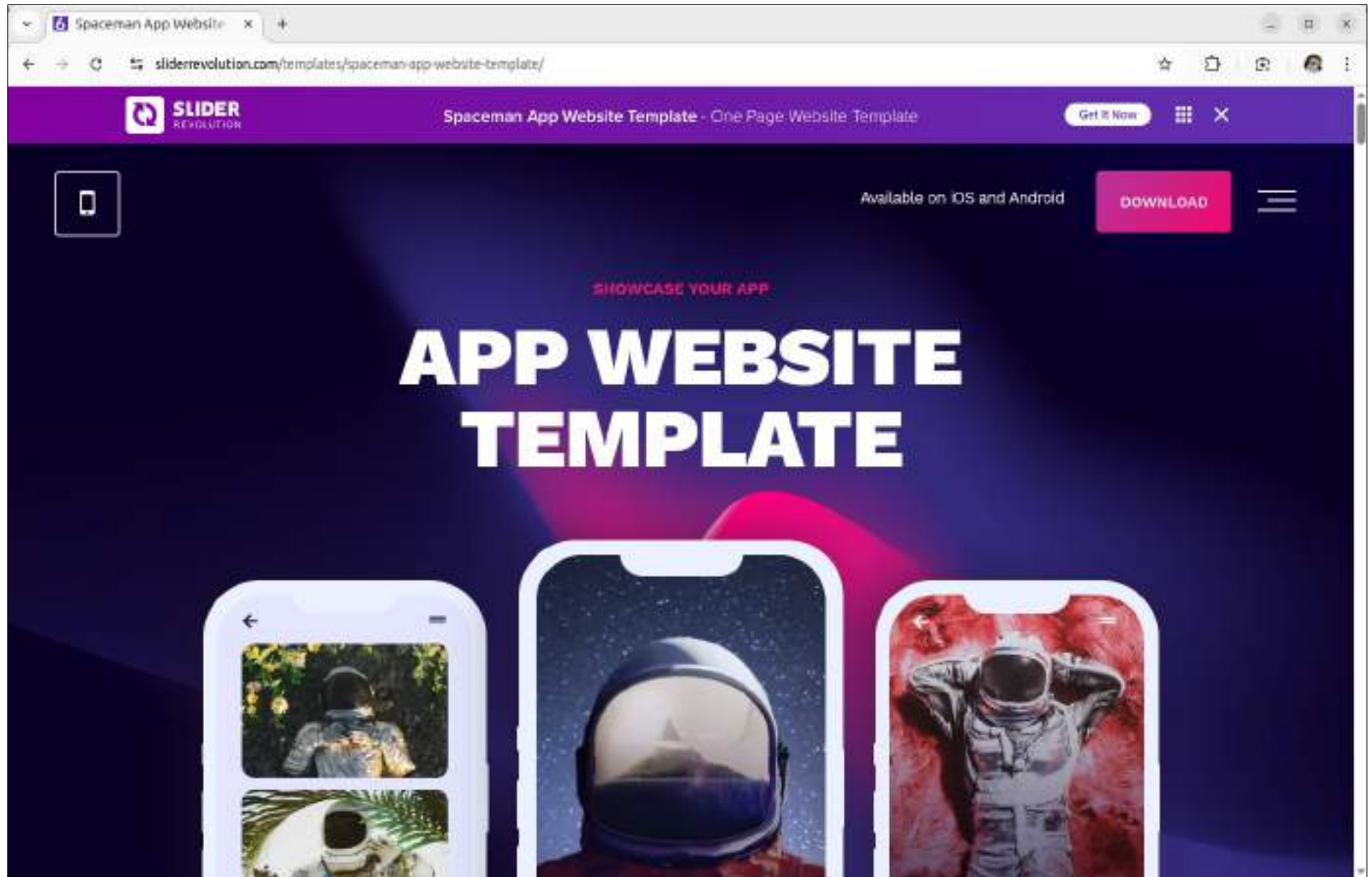
Servidor estático

- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA



<https://www.sliderrevolution.com/templates/spaceman-app-website-template/>

Taxonomía de las webs

- **Página web**



- Se utilizan **librerías de componentes CSS**



tailwindcss



Bootstrap

Página web

Servidor estático

- Página web

Aplicación web

Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

Tecnologías de servidor



Inicio - Universidad Rey

urjc.es

URJC

INICIOEMPRESASPRENSASIDE ELECTRÓNICACONTACTOAYUDAINTERNETTIENDA ONLINE

BUSCAR

Universidad
Rey Juan Carlos

UNIVERSIDADESTUDIOSESTUDIAR EN LA URJC I+D+iINTERNACIONALACTUALIDAD

HERRAMIENTAS DE
ACCESIBILIDAD

Feria Madrid
es ciencia 2026

del 19 al 21 de marzo

El Museo Universidad de Navarra reconoce a Liuba González Cid

19-03-2026

Ángel Velázquez Iturbide, miembro del ACM Europe Awards Group

18-03-2026

Inaugurado el Bloomberg Lab en el campus de Madrid

18-03-2026

La URJC ha implantado oficialmente su

Lleno total en la jornada sobre empleo en salud

18-03-2026

La Universidad Rey Juan Carlos ha

create a new account

Your Email

test@test



Please use a valid email address.

Confirm Email

Country

United States



Zip Code

123



Please provide a valid Zip Code

Password



Your password must be between 6-16 characters long.

Confirm Password



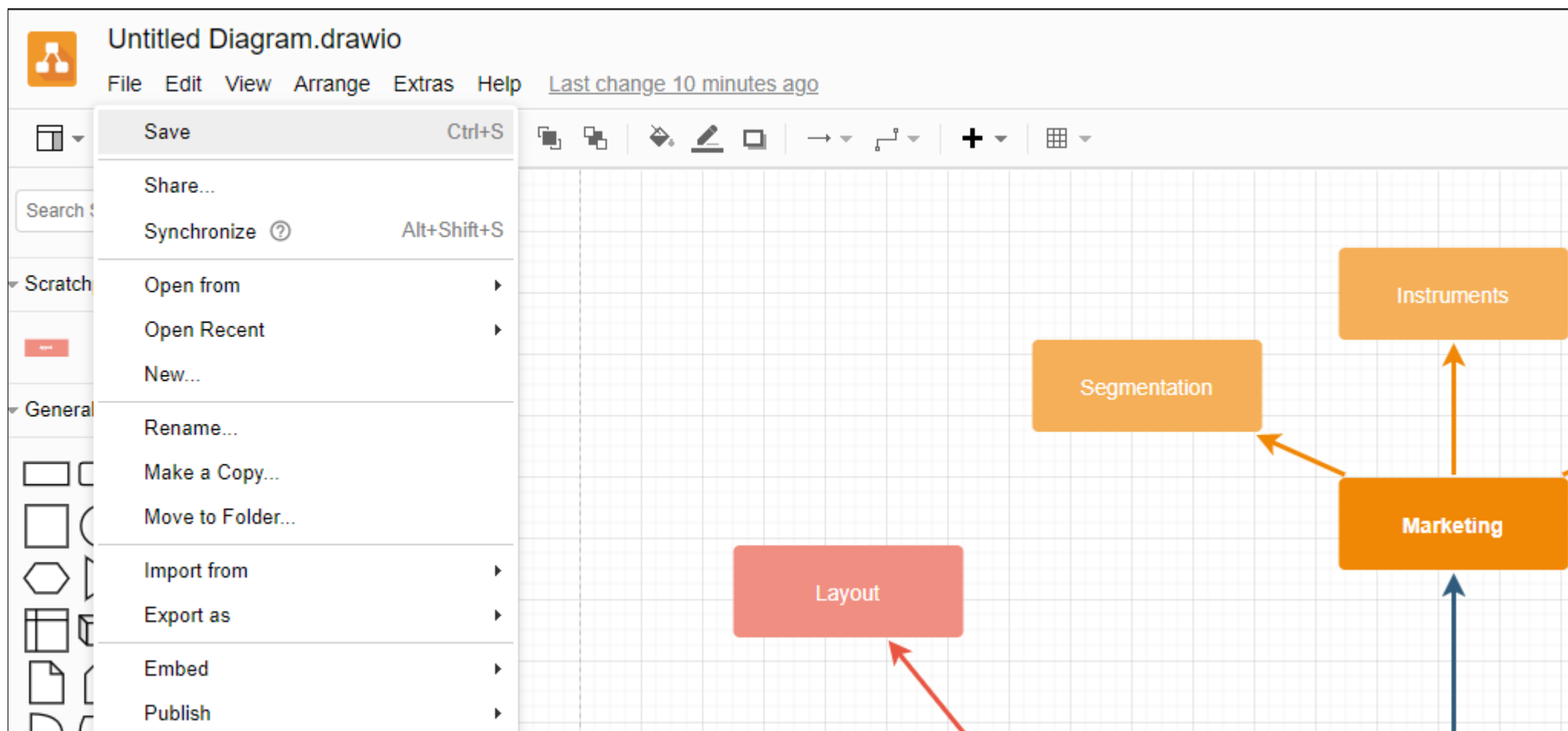
This does not match the password entered above.

☐ Yes, I agree to the [Terms of Use](#)

Sign Up

Why you'll love it

- See all your accounts in one place
- Set a budget and pay down your debt
- Find the best ways to grow your money
- Stay safe and secure



Página web

Servidor estático

- Página web

Aplicación web

Servidor dinámico

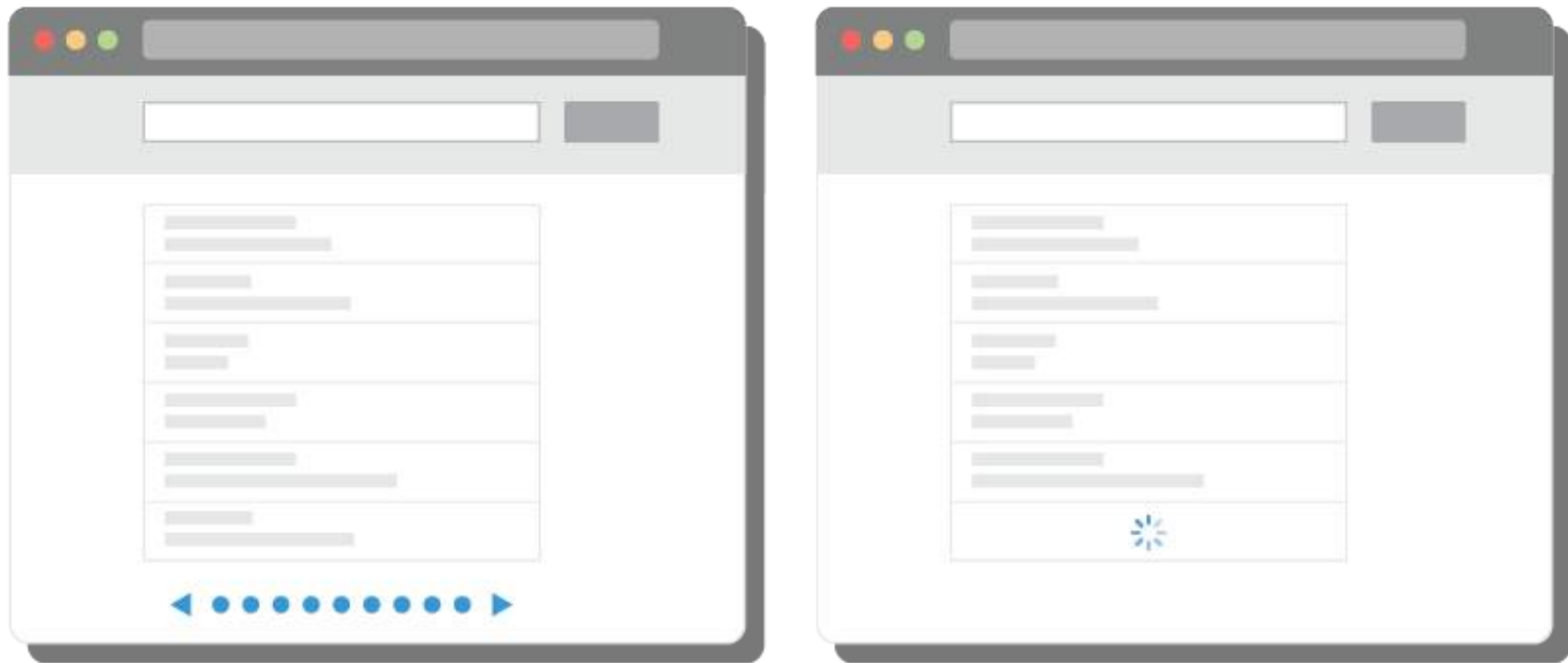
- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

- **Aplicación web con AJAX**
 - Comunicación con el servidor en segundo plano (sin recarga completa de página)



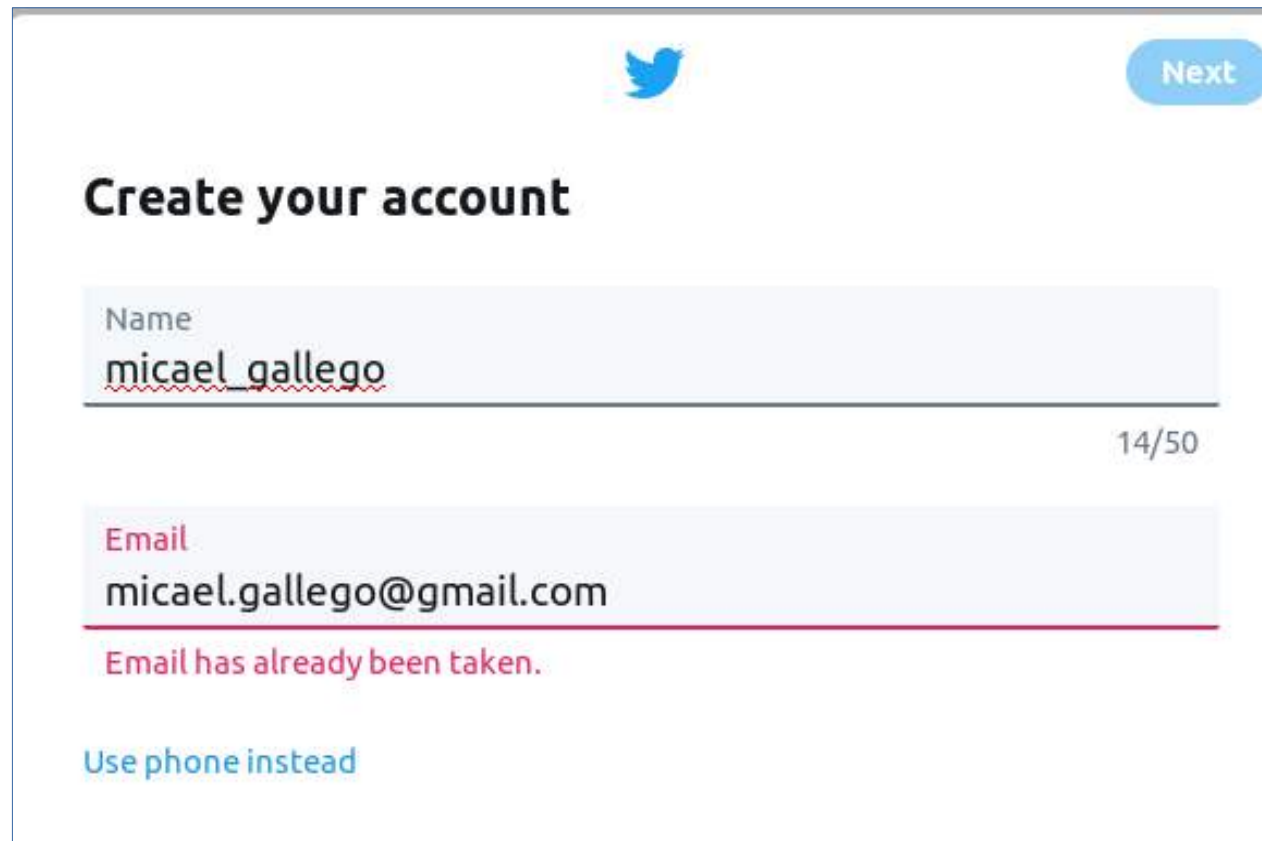
Taxonomía de las webs

- Aplicación web con AJAX



Taxonomía de las webs

- Aplicación web con AJAX



The screenshot shows a Twitter account creation interface. At the top, there is a Twitter bird logo and a blue 'Next' button. The main heading is 'Create your account'. Below this, there are two input fields. The first field is labeled 'Name' and contains the text 'michael_gallego', which is underlined with a red wavy line indicating a validation error. To the right of this field, the text '14/50' is displayed. The second field is labeled 'Email' and contains the text 'michael.gallego@gmail.com', also underlined with a red wavy line. Below the email field, a red error message states 'Email has already been taken.' At the bottom of the form, there is a blue link that says 'Use phone instead'.

Página web

Servidor estático

- Página web

Aplicación web

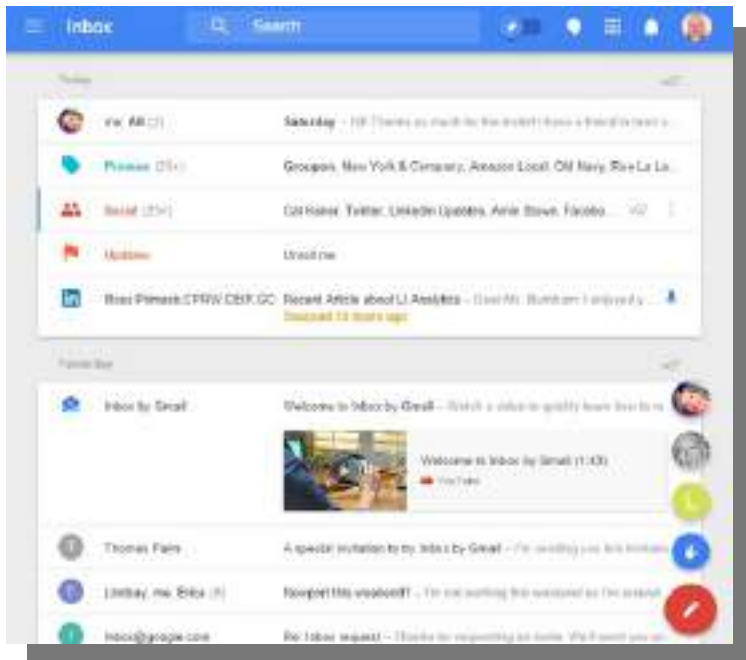
Servidor dinámico

- Aplicación web
- Aplicación web con AJAX
- Aplicación web SPA

- **Aplicación web SPA**
 - La página es un HTML casi vacío con JavaScript
 - El JavaScript construye la interfaz cargando datos del servidor (en segundo plano)
 - Mejor experiencia del usuario

Taxonomía de las webs

- Aplicación web SPA



1

<http://www.miweb.com/>



2

<http://api.miweb.com/users/34>

```
{  
  "name": "Pepe",  
  "surname": "López",  
  "age": 45,  
  "email": "pepe@miweb.com",  
  "friends": [ "María", "Juan" ]  
}
```



Google Maps

Universidad Rey Juan Carlos - Google Maps - Mozilla Firefox

Universidad Rey Juan Car... x

https://www.google.es/maps/place/Universidad+Rey+Juan+Carlos/@40.3347481,-3.8755547, Buscar

an Carlos, Calle Tulipán, Móstoles

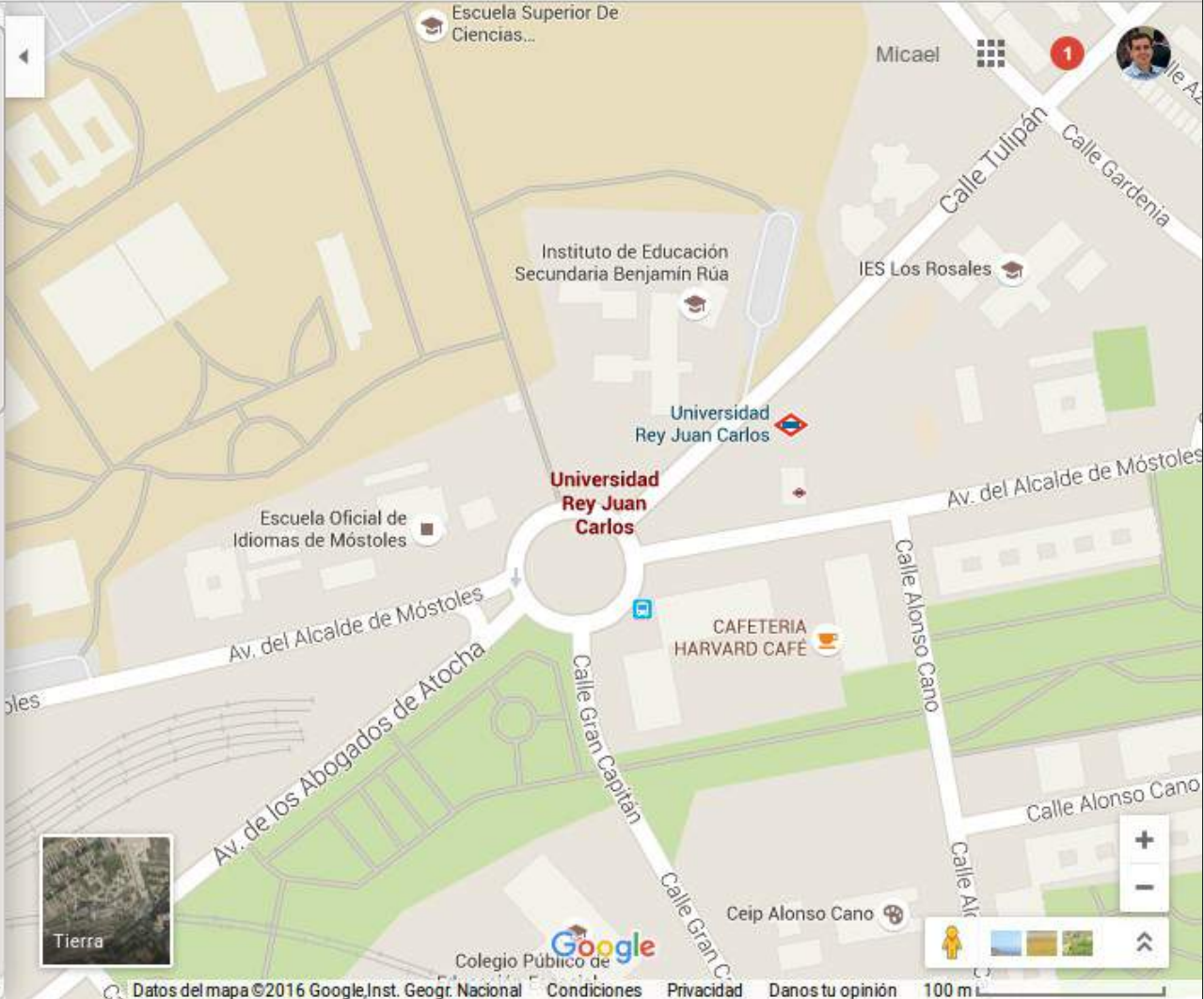


Universidad Rey Juan Carlos
3,8 ★★★★★ 66 reseñas
Universidad

Cómo llegar

GUARDAR LUGARES CERCANOS ENVIAR A TU TELÉFONO COMPARTIR

Calle Tulipán, s/n, 28933 Móstoles, Madrid
urjc.es
916 65 50 60
Cerrado hoy



Escuela Superior De Ciencias...

Micael

Calle Tulipán

Calle Gardenia

Instituto de Educación Secundaria Benjamín Rúa

IES Los Rosales

Universidad Rey Juan Carlos

Escuela Oficial de Idiomas de Móstoles

Av. del Alcalde de Móstoles

CAFETERIA HARVARD CAFÉ

Calle Alonso Cano

Calle Gran Capitán

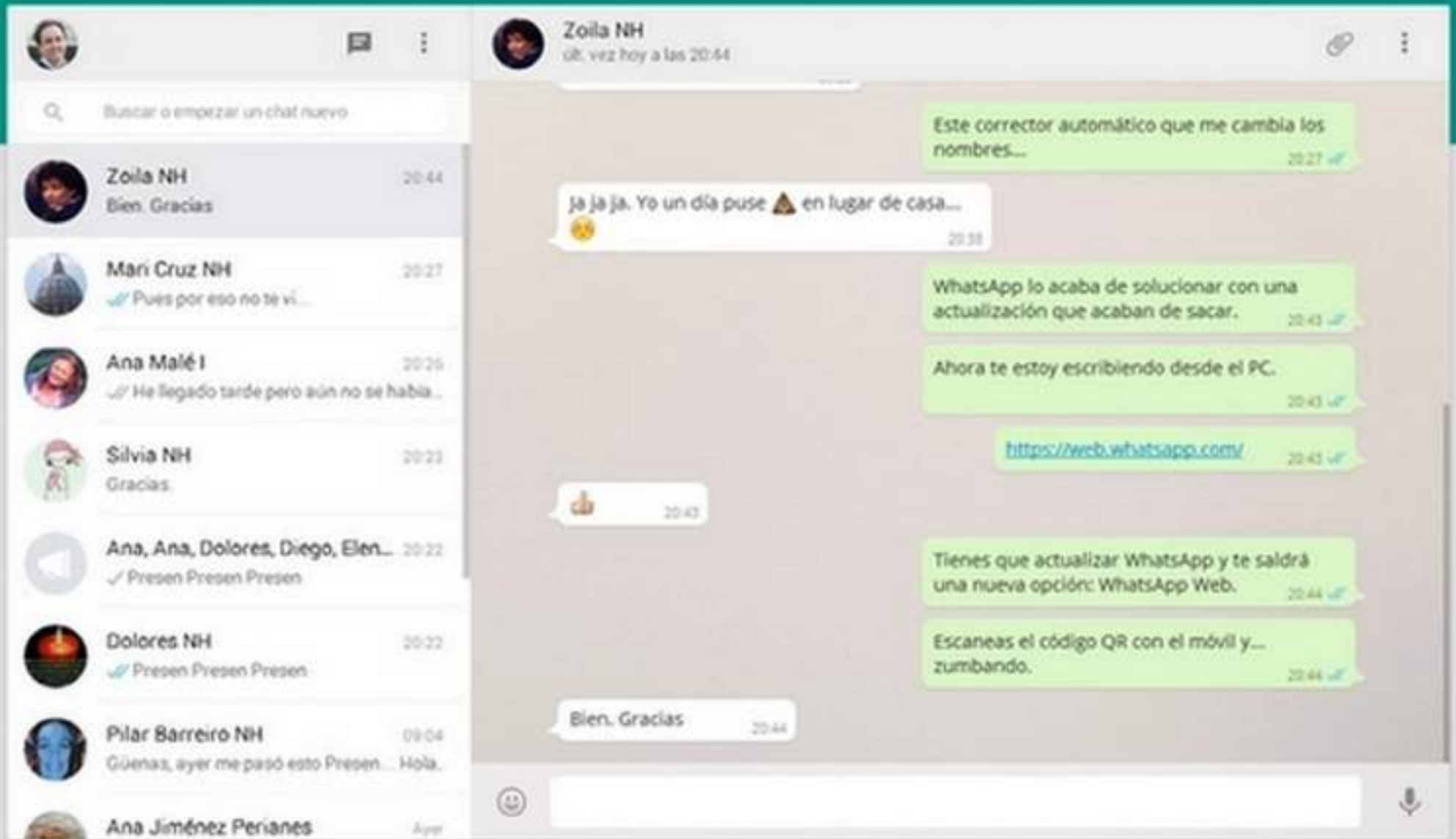
Calle Alonso Cano

Ceip Alonso Cano

Colegio Público de

Datos del mapa ©2016 Google, Inst. Geogr. Nacional Condiciones Privacidad Danos tu opinión 100 m

WhatsApp



GitHub

The screenshot shows the GitHub repository page for 'projecto26/awesome-jsgames'. The repository is public and has 837 stars, 77 forks, and 20 watchers. The main branch is 'main'. The repository contains a README.md file, a CONTRIBUTING.md file, a LICENSE file, and an 'img' directory. The repository is maintained by 'jdnichollsc' and has 96 commits. The repository is licensed under CC0-1.0 license. The repository is also featured on the 'Awesome JavaScript Games' list.

projecto26 / awesome-jsgames

Code Issues 1 Pull requests 2 Actions Projects Security Insights

awesome-jsgames Public

Sponsor Watch 20 Fork 77 Star 837

main 1 Branch 0 Tags

Go to file

Code

About

A curated list of awesome JavaScript Games 🎮

open-source games game-development js13k js13kgames awesome-list html5-game-development javascript-game js-games javascript-games javascript-game-engine html5-games games-list awesome-lists game-engines html5-game-engine js13kgames-starter web-monetization web-games

Readme

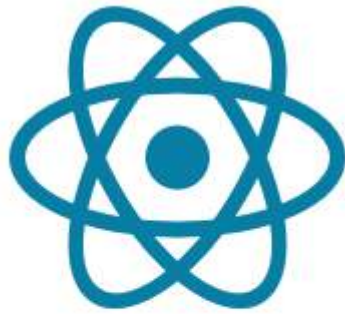
CC0-1.0 license

Contributing

Maintained? yes website up Follow @jdnichollsc

Awesome JavaScript Games 🎮

Frameworks / librerías SPA



React

<https://react.dev/>

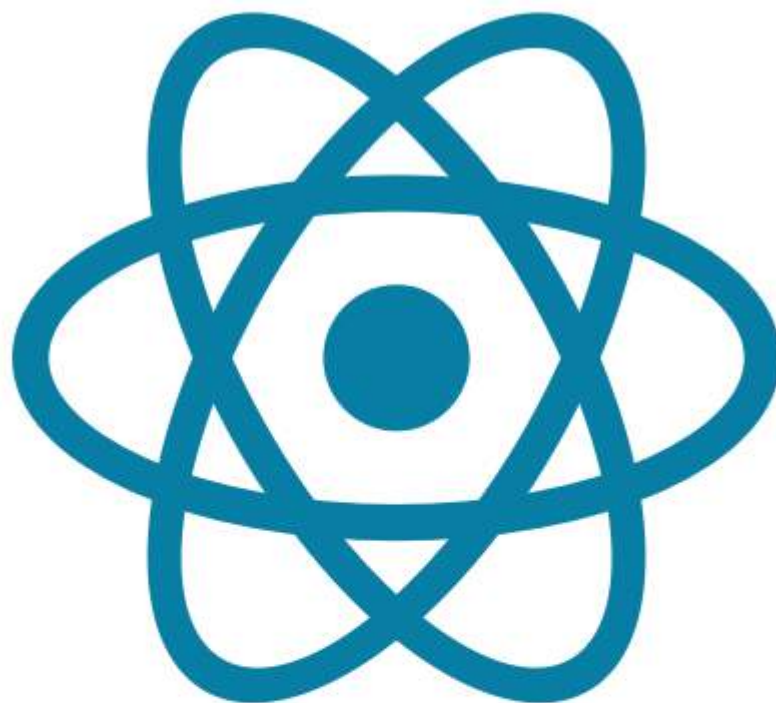


VUE.JS

<https://vuejs.org/>



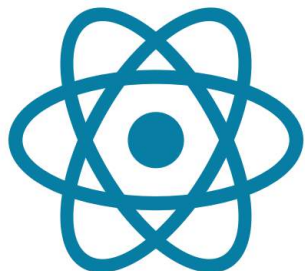
<https://angular.dev/>



React

<https://react.dev/>

- React es una librería para crear **webs dinámicas**
- Permite extender el **HTML con etiquetas propias**
 - Con contenido personalizado (HTML)
 - Con comportamiento personalizado (JavaScript)
- Interfaz basado en **componentes** (no en páginas)
- Se programa con **JavaScript** o **TypeScript** (una variante de JavaScript con tipado)



<https://react.dev/>



Características

- Generación de HTML partiendo de datos en variables JavaScript sin gestionar el DOM a bajo nivel
- Composición de componentes
- Compartir datos entre componentes
- Apps móviles (React Native)
- Es una librería muy ligera que se puede usar de muchas formas diferentes
- Hay un gran ecosistema alrededor

- React es una **librería** para añadir interactividad a una página
- Para crear aplicaciones SPA es necesario un **framework** para:
 - Gestión de páginas (rutas)
 - Gestiona la carga de datos de API REST
 - Generación del esqueleto del proyecto
 - Previsualización de cambios en el código al instante (*hot reloading*)

Frameworks React



NEXT.JS



React Router

- **Framework** basado en React
- Simula páginas (rutas) en una aplicación SPA
- Gestiona la carga de datos de API REST
- Genera el esqueleto del proyecto
- Previsualiza los cambios en el código al instante (*hot reloading*) mediante Vite

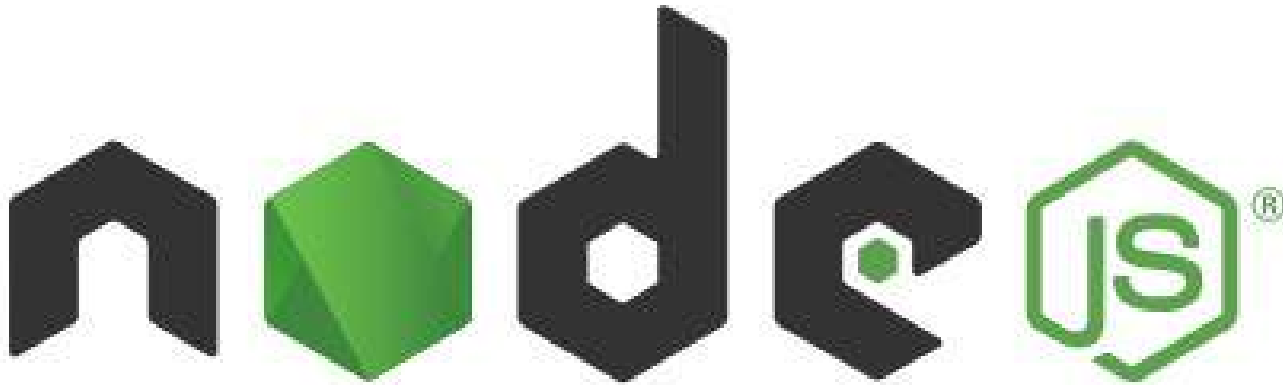


<https://reactrouter.com/>

Herramientas

Para desarrollar aplicaciones SPA

React Router



Download Node.js (LTS) 

<https://nodejs.org/>

React Router CLI

- Genera el **esqueleto** de una aplicación
- Permite añadir **componentes, servicios..**
- Permite **actualizar** código y previsualizar
- **Modo desarrollo:** Editar código y actualizar el navegador
- **Modo producción:** Optimiza el código para ejecución y rápida y poco tamaño (*build*)

<https://reactrouter.com/api/other-api/dev>

React Router

- Crear un proyecto

```
npx create-react-router@latest exam0_minimal --template remix-run/react-router-templates/minimal
```

```
• mica@mica-laptop:~/git/react$ npx create-react-router@latest ejem0 --template remix-run/react-router-templates/minimal

  create-react-router v7.13.0
  ■ Directory: Using ejem0 as project directory
  ■ Template: Using remix-run/react-router-templates/minimal...
  ✓ Template copied

  git Initialize a new git repository?
  Yes

  deps Install dependencies with npm?
  Yes

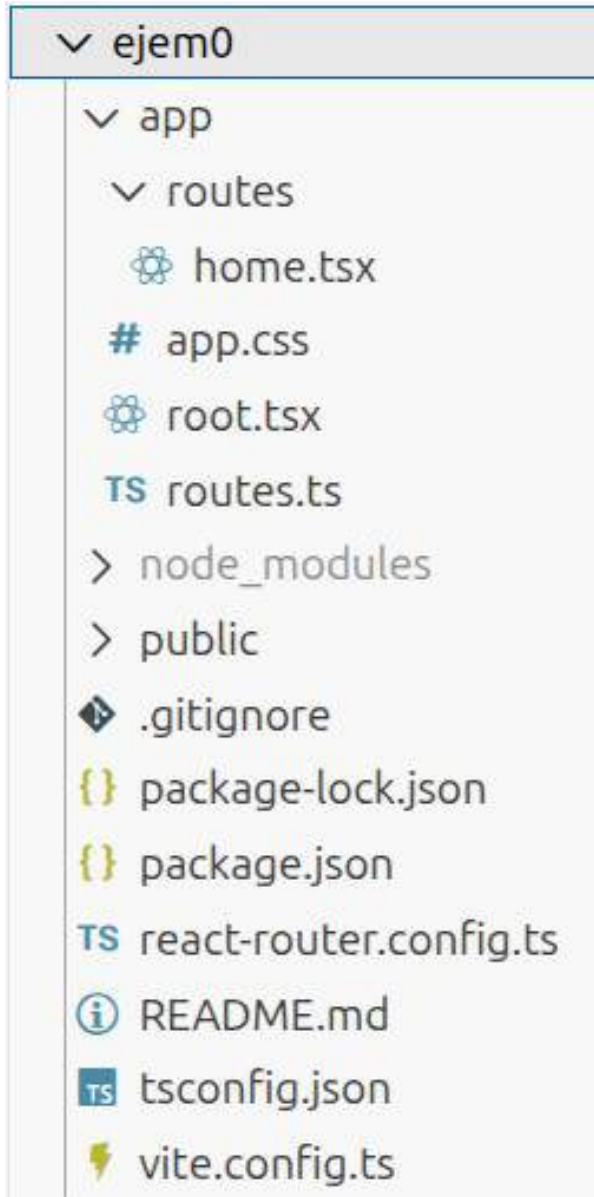
  ✓ Dependencies installed
  ✓ Git initialized

  done That's it!

  Enter your project directory using cd ./ejem0
  Check out README.md for development and deploy instructions.

  Join the community at https://rmx.as/discord
```

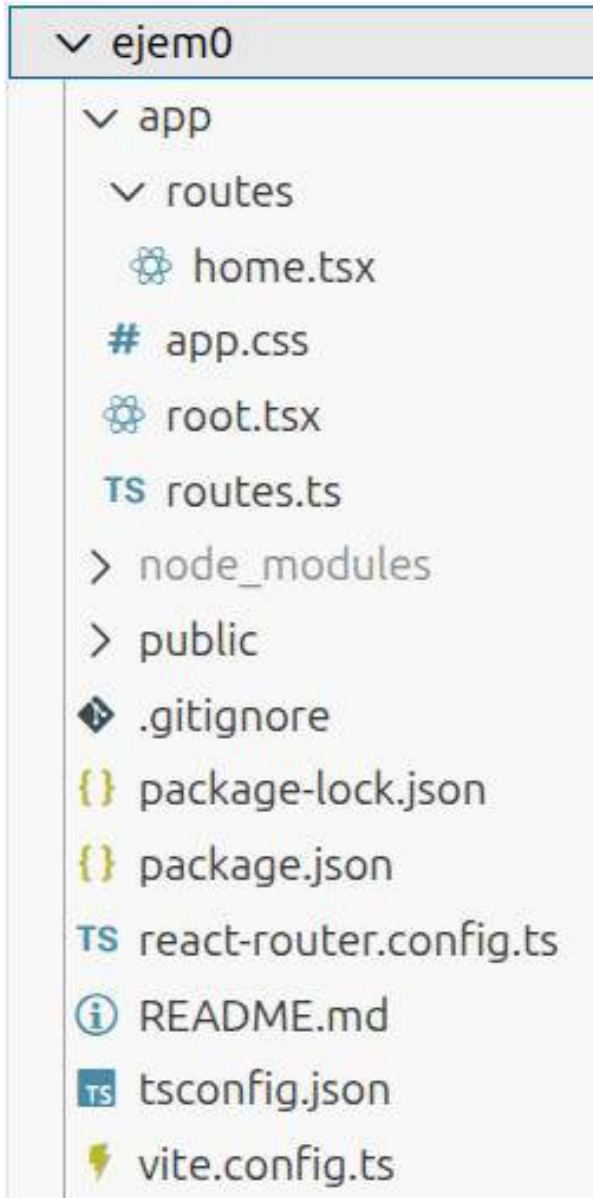
Un proyecto ocupa **108 MB** en disco



- **Ficheros/Carpetas generadas**

- app:** Código de la aplicación

- **public:** recursos estáticos web (imágenes...)
- **node_modules:** Librerías y herramientas descargadas
- **package.json:** Configuración de librerías Node y herramientas
- **react-router.config.json:** Configuración del framework
- **vite.config.ts:** Configuración de vite
- **tsconfig.json:** Configuración del compilador TypeScript



- **Ficheros/Carpetas generadas**
 - Ficheros que vamos a editar al principio
 - **home.tsx**: Código TypeScript app react
 - **app.css**: Estilos globales de la app
 - **public**: Recursos públicos (imágenes, etc.)
 - Ficheros que no vamos a editar (salvo casos puntuales)
 - **root.tsx**: Fichero principal de la aplicación
 - **react-router.config.ts**
 - **vite.config.ts**

Visual Studio Code

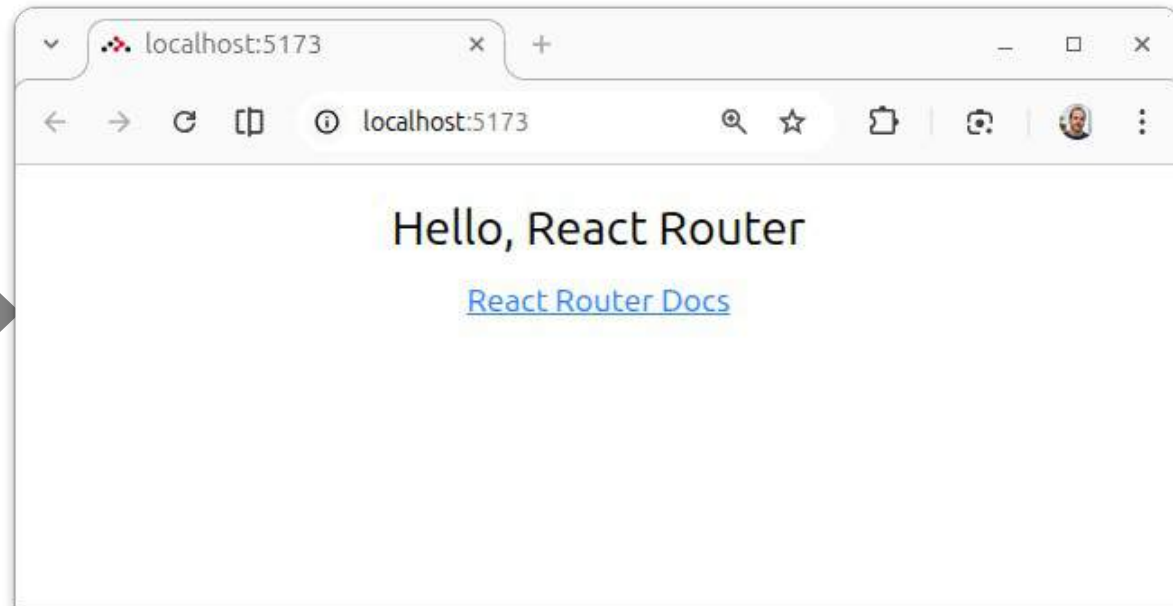


A screenshot of the Visual Studio Code interface. The main editor window displays a TypeScript file named MailList.tsx. The code includes imports for solid-js, solid-router, and a custom MailListItem component, followed by a Maillist function and an open function. The left sidebar shows the 'AGENT SESSIONS' panel with three sessions: 'LOCAL CHAT AGENT', 'GITHUB COPILOT CLOUD AGENT' (active), and 'GITHUB COPILOT CLI AGENT'. The 'GITHUB COPILOT CLOUD AGENT' session shows a task 'Extract Maillistitem into st...' with a duration of 10 minutes and a status of #1: +42 -24. The right sidebar contains a 'CHAT' panel with a message from the GitHub Copilot Cloud Agent: 'Please update the Maillist component to abstract mail list items into their own standalone component.' Below this, there are sections for 'GitHub Copilot Cloud Agent' with a 'Delegate to cloud agent' button and a description of the agent's asynchronous work. Further down, it shows a user 'kenzi.lawson' selected 'Delegate', followed by another 'GitHub Copilot Cloud Agent' section with a '[WIP] Refactor Maillist to abstract mail list items - Copilot' task. At the bottom, it states 'Cloud agent has begun work on Please update the Ma... and will update t' with a 'See more' link. The bottom of the chat panel shows 'GitHub Copilot cloud agent has begun working on your request. Follow its progress in the' and a 'Maillist.tsx +' button. The bottom status bar shows 'Agent: Claude-Opus 4.5'.

<https://code.visualstudio.com>

- Empezar a desarrollar

```
$ cd exam0_minimal  
$ npm run dev
```



<http://localhost:5173/>

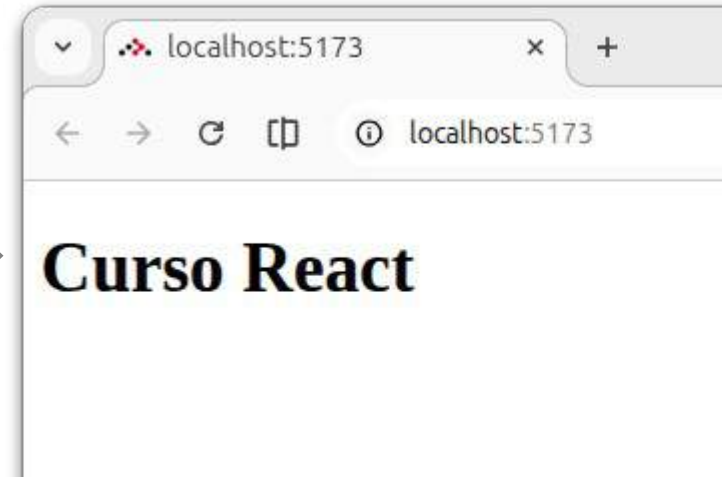
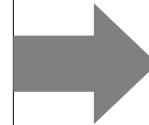
- Ejecutar la app en modo desarrollo

```
npm run dev
```

Al cambiar un fichero fuente y guardar, la aplicación se **actualiza automáticamente**

exam0_minimal/app/routes/home.tsx

```
export default function Home() {  
  return (  
    <h1>Curso React</h1>  
  );  
}
```



- Ejecutar el ejemplos y ejercicios
 - Descargar el fichero .zip
 - Descomprimir en una carpeta y abrir en VSCode
 - Por cada proyecto
 - Descarga las librerías del proyecto

```
$ cd exam1_render_var  
$ npm install
```

- Ejecutar la aplicación en modo desarrollo

```
$ npm run dev
```


TypeScript

- React se puede programar con **JavaScript** o **TypeScript**
- En los proyectos **React Router**, por defecto se usa TypeScript
- Es una variante de JavaScript cada vez más **popular**



Características

- Variante de JavaScript con **validación de tipos**
- Se usa cada vez más por la ayuda del IDE (**autocompletar y avisa de errores**)
- Integrado en todos los **editores / IDEs**

<http://www.typescriptlang.org/>



```
export class Empleado {                                     TypeScript

    private nombre:string;
    private salario:number;

    constructor(nombre:string, salario:number){
        this.nombre = nombre;
        this.salario = salario;
    }

    getNombre(){
        return this.nombre;
    }

    toString(){
        return "Nombre:"+this.nombre+
            ", Salario:"+this.salario;
    }
}
```

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En Java las clases son públicas. En TypeScript las clases se exportan

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS los tipos se ponen tras el nombre con : (dos puntos)

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS el constructor es "constructor"

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:"+nombre+  
            ", Salario:"+salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:"+this.nombre+  
            ", Salario:"+this.salario;  
    }  
}
```

En TS los tipos se infieren (si quieres), incluso en el tipo que devuelven los métodos

Clases Java vs TypeScript

Clase en Java

```
public class Empleado {  
  
    private String nombre;  
    private double salario;  
  
    public Empleado(String nombre,  
        double salario){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    public String getNombre(){  
        return nombre;  
    }  
  
    public String toString(){  
        return "Nombre:" + nombre +  
            ", Salario:" + salario;  
    }  
}
```

Clase en TypeScript

```
export class Empleado {  
  
    private nombre:string;  
    private salario:number;  
  
    constructor(nombre:string,  
        salario:number){  
        this.nombre = nombre;  
        this.salario = salario;  
    }  
  
    getNombre(){  
        return this.nombre;  
    }  
  
    toString(){  
        return "Nombre:" + this.nombre +  
            ", Salario:" + this.salario;  
    }  
}
```

En TS siempre que quieras acceder a un elemento de la clase tendrás que usar this.

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "./Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "./Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

En Java las clases de la misma carpeta son del mismo paquete.
En TS cada fichero es un módulo, por eso hay que importar desde otros ficheros

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En TS las variables se declaran con `let` y tienen ámbito de bloque y no se pueden declarar dos veces. Se podría usar `var` (como JS), pero el ámbito sería la función y se podrían redeclarar

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En TS las variables se pueden declarar con tipo (después de :),
pero es opcional porque el tipo se infiere del valor con el que se inicializan

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();
```

```
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));
```

```
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}
```

```
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";
```

```
let emps = new Array<Empleado>();
```

```
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));
```

```
for(let emp of emps){  
    console.log(emp.getNombre());  
}
```

```
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En Java usamos List y ArrayList generificados del API.
En TS usamos el Array nativo de JavaScript generificado por TS.

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();  
emps.add(new Empleado('Pepe', 500));  
emps.add(new Empleado('Juan', 200));  
  
for(Empleado emp : emps){  
    System.out.println(emp.getNombre());  
}  
  
empleados.forEach(emp -> {  
    System.out.println(emp);  
});
```

TypeScript

```
import { Empleado } from "../Empleado";  
  
let emps = new Array<Empleado>();  
emps.push(new Empleado('Pepe', 500));  
emps.push(new Empleado('Juan', 200));  
  
for(let emp of emps){  
    console.log(emp.getNombre());  
}  
  
empleados.forEach(emp => {  
    console.log(emp);  
});
```

En Java List el método es "add"
En el array de JavaScript el método es "push"

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "../Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

La sintaxis del foreach es muy parecida en Java y TS.
En TS está basado en iteradores (como en Java)

Imports / Listas / foreach / lambdas

Java

```
List<Empleado> emps = new ArrayList<>();

emps.add(new Empleado('Pepe', 500));
emps.add(new Empleado('Juan', 200));

for(Empleado emp : emps){
    System.out.println(emp.getNombre());
}

empleados.forEach(emp -> {
    System.out.println(emp);
});
```

TypeScript

```
import { Empleado } from "../Empleado";

let emps = new Array<Empleado>();

emps.push(new Empleado('Pepe', 500));
emps.push(new Empleado('Juan', 200));

for(let emp of emps){
    console.log(emp.getNombre());
}

empleados.forEach(emp => {
    console.log(emp);
});
```

Las expresiones lambda de Java se llaman *arrow functions* en TS.
Se diferencian en la "flecha" con – o con =

Objetos literales “tipados”

```
interface SquareConfig {  
  color: string;  
  width?: number;  
}
```

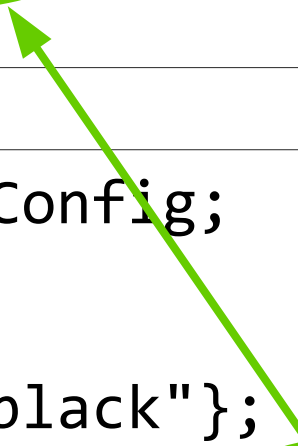
```
let config: SquareConfig;  
  
config = {color: "black"};  
config = {color: "black", width: 20};
```

Objetos literales “tipados”

```
interface SquareConfig {  
  color: string;  
  width?: number;  
}
```

```
let config: SquareConfig;
```

```
config = {color: "black"};  
config = {color: "black", width: 20};
```



Tipos unión

```
let id: string | number;  
  
id = 3;  
...  
Id = "3";  
  
if(typeof id === "string"){  
    ...  
}
```

TypeScript permite que una variable tenga valores de diferentes tipos